Credit Card Recommendation System (CCRS)

Version: 1.0

Document Status: Final

Project Type: Web Application

Architecture: Deterministic Rule-Based Recommendation System

---

1. Executive Summary

1.1 Overview

The **Credit Card Recommendation System (CCRS)** is a web-based application designed to simplify the process of selecting an appropriate credit card by recommending the most suitable cards based on a user's preferences, spending habits, and desired benefits.

Modern consumers are presented with hundreds of credit card options across multiple financial institutions. Comparing reward structures, annual fees, travel benefits, cashback offers, insurance coverage, lounge access, and eligibility requirements manually is time-consuming and often confusing. CCRS addresses this problem by allowing users to complete a structured questionnaire that captures their preferences through a checklist-based interface. The application evaluates all supported credit cards using a deterministic rule-based scoring engine and returns the most relevant recommendations together with transparent explanations.

Unlike AI-powered recommendation platforms, CCRS does not use Artificial Intelligence, Machine Learning, Large Language Models, or external recommendation services. Every recommendation is generated using predefined scoring rules stored within a local SQLite database. The recommendation process is fully deterministic, ensuring that identical user inputs always produce identical outputs.

The application shall be designed as a scalable web platform capable of supporting approximately **10,000 users per day** while maintaining consistent performance and responsiveness.

1.2 Business Problem

Consumers typically compare credit cards manually across multiple banking websites.

This creates several challenges.

- Large number of available credit cards
- Different reward structures
- Complex fee structures
- Changing promotional offers
- Different eligibility criteria
- Difficult comparison process

The objective of CCRS is to reduce this complexity by allowing users to describe their requirements instead of manually comparing every available card.

1.3 Business Goals

The project aims to achieve the following objectives.

- Simplify the credit card selection process.
- Reduce manual comparison effort.
- Provide deterministic recommendations.
- Ensure complete recommendation transparency.
- Maintain reproducible recommendation results.
- Allow future expansion to additional banks.
- Support enterprise-grade scalability.
- Provide an intuitive and responsive user experience.

1.4 Project Scope

The project includes the complete development of a web-based recommendation platform consisting of:

- Responsive Frontend
- Backend REST API
- SQLite Database
- Rule-Based Recommendation Engine
- Search Module
- Filter Module

- Card Comparison Module
- Individual Card Detail Pages
- Docker Deployment
- Documentation

The project does not include:

- Online credit card applications
- Banking integrations
- Payment processing
- User credit score calculation
- Financial advice
- AI-based recommendations
- Machine learning
- Third-party recommendation APIs

---

2. Vision & Objectives

2.1 Product Vision

The Credit Card Recommendation System shall function as a deterministic recommendation engine rather than an intelligent assistant.

Users answer a structured questionnaire.

The backend evaluates every supported credit card using configurable scoring rules stored within SQLite.

The system recommends the highest-scoring cards together with a detailed explanation of why they were selected.

Every recommendation shall be explainable, transparent, reproducible, and fully deterministic.

2.2 Product Objectives

The application shall:

- Recommend the most suitable credit cards.
- Provide explainable recommendations.

- Display recommendation scores.
- Display benefit coverage percentages.
- Allow side-by-side card comparison.
- Allow browsing all supported cards.
- Allow searching credit cards.
- Allow advanced filtering.
- Store all metadata within SQLite.
- Read recommendation rules dynamically from the database.

2.3 Success Metrics

The application shall be considered successful when:

- Users receive recommendations within two seconds under normal operating conditions.
- Identical questionnaire responses always produce identical recommendations.
- Every supported card is stored within SQLite.
- Recommendation explanations are generated correctly.
- Search functionality returns relevant cards.
- Filters operate correctly.
- Compare module functions correctly.
- Application remains responsive across desktop, tablet, and mobile devices.

---

3. Product Scope

3.1 Included Features

The initial release (Version 1.0) shall include:

- Recommendation Questionnaire
- Recommendation Engine
- Browse Credit Cards
- Individual Card Pages
- Compare Cards
- Search
- Filters
- SQLite Database
- REST API
- Responsive User Interface

- Docker Deployment

3.2 Excluded Features

The following functionality is explicitly excluded from Version 1.

- User account management
- Credit score analysis
- Online application submission
- Bank API integrations
- AI-generated recommendations
- Personalized financial advice
- Payment gateway integration
- Spending analytics
- Automatic card updates
- Mobile application

---

4. Target Audience

The application is intended for the following user groups.

First-Time Credit Card Applicants

Individuals applying for their first credit card who require assistance understanding available options.

Salaried Employees

Users seeking reward cards, cashback cards, travel cards, or lifestyle cards based on monthly spending patterns.

Business Owners

Business users looking for cards offering travel benefits, premium services, higher spending limits, and business-oriented rewards.

Students

Students searching for entry-level credit cards with lower eligibility requirements and minimal annual fees.

Frequent Travellers

Users interested in:

- Domestic Lounge Access
- International Lounge Access
- Air Miles
- Hotel Rewards
- Travel Insurance
- Concierge Services

Cashback Users

Individuals prioritizing cashback across:

- Online Shopping
- Grocery
- Fuel
- Utility Payments
- Dining
- Digital Payments

Lifestyle Users

Individuals primarily interested in:

- Movie Discounts

- Dining Offers
- Golf Privileges
- Premium Memberships
- Entertainment Benefits

Users Comparing Credit Cards

Individuals who already have shortlisted cards and require a detailed side-by-side comparison before making a decision.

---

5. User Personas

Persona 1 – Rahul (Young Professional)

Age: 25

Occupation: Software Engineer

Income: ₹8 LPA

Goals

- Cashback
- Fuel Benefits
- Low Annual Fee
- UPI Support

Pain Points

- Too many similar cards
- Difficult comparison
- Hidden charges

Persona 2 – Priya (Frequent Traveller)

Age: 31

Occupation: Consultant

Goals

- Domestic Lounge
- International Lounge
- Air Miles
- Travel Insurance
- Hotel Rewards

Pain Points

- Difficult reward comparisons
- Multiple travel cards
- Complicated reward structures

Persona 3 – Amit (Business Owner)

Goals

- High Reward Rate
- Concierge
- Golf Benefits
- Premium Travel Benefits

Pain Points

- Complex premium card comparison
- High annual fees
- Unclear eligibility requirements

---

6. Supported Banks

Version 1 of the application shall support consumer credit cards offered by the following financial institutions:

- HDFC Bank
- SBI Card

- Axis Bank
- AU Small Finance Bank
- American Express India
- Bank of Baroda

The application architecture shall be data-driven.

Adding new banks shall require only database updates and shall not require modifications to the recommendation engine or application source code.

---

7. Functional Requirements

7.1 Overview

The application shall consist of the following major functional modules.

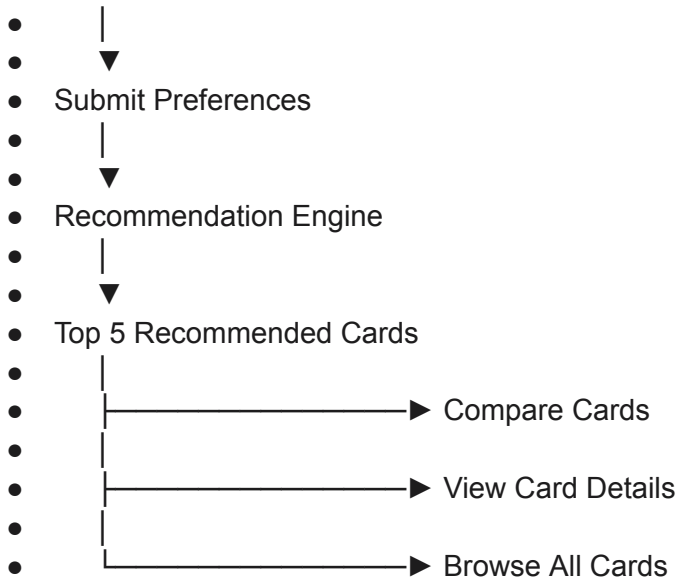
1. Recommendation Questionnaire
2. Recommendation Engine
3. Recommendation Results
4. Browse Credit Cards
5. Individual Card Details
6. Compare Cards
7. Search
8. Filters
9. SQLite Database
10. REST API

Every module shall operate independently while interacting through clearly defined backend services.

7.2 User Journey

The standard user flow shall be:

- Home Page
- │
- ▼
- Recommendation Questionnaire



7.3 Recommendation Questionnaire

The homepage shall present a structured questionnaire using grouped checkbox selections.

Users may select:

- A single requirement
- Multiple requirements
- Every available requirement

Questionnaire categories shall include:

- Rewards
- Travel
- Shopping
- Lifestyle
- Digital Payments
- Fuel
- Insurance
- Fees
- Eligibility
- Reward Redemption
- Other Features

Each category shall contain multiple selectable benefits. The questionnaire shall support future expansion without requiring frontend redesign.

8. Detailed Functional Requirements

8.1 Recommendation Engine

The Credit Card Recommendation System shall utilize a deterministic rule-based recommendation engine that evaluates every supported credit card against the user's selected preferences.

The recommendation engine shall not use Artificial Intelligence, Machine Learning, Large Language Models, Neural Networks, predictive analytics, or external recommendation services. All recommendation logic shall be executed using configurable weighted scoring rules stored within the SQLite database.

The recommendation workflow shall consist of the following steps:

1. Receive user-selected preferences from the questionnaire.
2. Validate the submitted request.
3. Retrieve all eligible credit cards from the SQLite database.
4. Retrieve all scoring weights associated with each benefit.
5. Compare every selected benefit against every supported credit card.
6. Calculate weighted scores for every card.
7. Rank all cards according to their total score.
8. Apply tie-breaking rules where required.
9. Calculate benefit coverage percentages.
10. Generate transparent recommendation explanations.
11. Return the highest-ranked recommendations.

Recommendations shall always remain deterministic. The same questionnaire responses shall always generate identical recommendation results.

8.2 Recommendation Scoring

Each benefit stored within the database shall contain a configurable scoring weight.

Example scoring:

Benefit	Weight

Cashback	10
Airport Lounge	15
Fuel Benefits	8
Dining Offers	6
Shopping Offers	7
Travel Insurance	12
Forex Benefits	10
Golf Privileges	9

The backend shall retrieve these values dynamically from SQLite.

No scoring values shall be hardcoded within the application source code.

8.3 Recommendation Formula

The recommendation score shall be calculated using weighted benefit matching.

For every selected benefit:

- Matching benefit → Add configured weight.
- Missing benefit → Add zero points.

Final recommendation score shall equal the sum of all matched benefit weights.

Benefit Coverage Percentage shall be calculated as:

Matched Benefits ÷ Selected Benefits × 100

8.4 Tie-Breaking Rules

If multiple credit cards receive identical recommendation scores, the backend shall rank cards using the following priority:

1. Highest Recommendation Score
2. Highest Number of Matched Benefits
3. Highest Benefit Coverage Percentage
4. Lowest Annual Fee
5. Lowest Joining Fee
6. Highest Reward Rate
7. Alphabetical Card Name (final deterministic tie-breaker)

8.5 Recommendation Results

The recommendation page shall display the Top 5 highest-ranked credit cards.

Each recommendation shall display:

- Card Image
- Card Name
- Issuing Bank
- Recommendation Score
- Match Percentage
- Matched Benefits
- Missing Benefits
- Annual Fee
- Joining Fee
- Fee Waiver Conditions
- Reward Rate
- Cashback Categories
- Lounge Access
- Insurance Benefits
- Fuel Benefits
- Shopping Benefits
- Travel Benefits
- Eligibility
- Income Requirement
- Pros
- Cons
- Recommendation Explanation

The recommendation explanation shall explicitly identify which selected benefits contributed to the card's ranking.

8.6 Compare Cards

Users shall be able to compare any two supported credit cards.

The comparison page shall display both cards side-by-side.

Comparison categories shall include:

- Card Image
- Bank
- Card Type
- Joining Fee
- Annual Fee
- Fee Waiver
- Eligibility
- Reward Rate
- Cashback
- Reward Points
- Lounge Access
- Fuel Benefits
- Dining Benefits
- Shopping Benefits
- Insurance
- Travel Benefits
- Forex Benefits
- UPI Benefits
- Concierge
- Golf
- Welcome Bonus
- Renewal Benefits
- Add-on Cards
- EMI Conversion
- Balance Transfer
- Merchant Offers
- Pros
- Cons
- Overall Recommendation Score

8.7 Search

Users shall be able to search using:

- Card Name
- Issuing Bank
- Card Category
- Benefit
- Annual Fee
- Joining Fee

Search shall be case-insensitive and support partial matches.

8.8 Filtering

Users shall be able to combine multiple filters simultaneously.

Supported filters include:

Bank

- HDFC Bank
- SBI Card
- Axis Bank
- AU Small Finance Bank
- American Express
- Bank of Baroda

Fees

- Lifetime Free
- No Joining Fee
- Low Annual Fee

Benefits

- Cashback
- Travel
- Lounge Access
- Shopping
- Fuel
- Dining

- Insurance
- Entertainment
- Forex
- UPI

Multiple filters shall operate using logical AND conditions.

---

9. Website Pages

The application shall include the following pages.

Home

Contains:

- Navigation Bar
- Hero Section
- Recommendation Questionnaire
- Quick Search
- Featured Credit Cards
- Footer

Recommendation Results

Displays:

- Top Recommendations
- Recommendation Scores
- Match Percentage
- Recommendation Explanation
- Compare Button
- View Details Button

Browse Credit Cards

Displays every supported credit card with:

- Card Image
- Card Name
- Bank
- Annual Fee
- Reward Type
- Quick Benefits

Supports:

- Pagination
- Sorting
- Search
- Filters

Individual Card Details

Displays complete information about a selected card including:

- Basic Information
- Fees
- Eligibility
- Rewards
- Cashback
- Lounge Access
- Insurance
- Travel Benefits
- Shopping Benefits
- Fuel Benefits
- Pros
- Cons

Compare Cards

Displays a responsive side-by-side comparison table.

About

Provides information regarding the purpose and methodology of the recommendation engine.

Contact

Provides contact details and project information.

---

10. User Interface Requirements

The application shall use a clean, responsive, and modern interface.

The UI shall include:

- Responsive Navigation
- Responsive Grid Layout
- Card-Based Design
- Mobile-Friendly Forms
- Responsive Comparison Tables
- Search Bar
- Filter Sidebar
- Breadcrumb Navigation
- Loading Indicators
- Error Messages
- Empty-State Screens

The interface shall function correctly on:

- Desktop
- Tablet
- Mobile

---

11. Database Design

SQLite shall be used as the primary database.

The application shall contain, at minimum, the following tables.

Banks

- Bank ID
- Bank Name
- Logo
- Website

Credit Cards

- Card ID
- Bank ID
- Card Name
- Card Type
- Joining Fee
- Annual Fee
- Eligibility
- Income Requirement

Benefits

- Benefit ID
- Category
- Benefit Name

Card Benefits

Maps every benefit to every supported card.

Recommendation Weights

Stores configurable scoring values.

Categories

Stores questionnaire categories.

The database shall be normalized to reduce redundancy.

Indexes shall be created on:

- Card Name
- Bank
- Benefit Category
- Benefit ID

---

12. REST API Specification

The backend shall expose RESTful APIs.

Minimum endpoints include:

Method	Endpoint	Description
GET	/banks	Retrieve supported banks
GET	/cards	Retrieve all cards
GET	/cards/{id}	Retrieve card details
POST	/recommend	Generate recommendations
POST	/compare	Compare selected cards
GET	/search	Search cards
GET	/filters	Retrieve available filters

All API responses shall use JSON.

---

13. Performance Requirements

The application shall:

- Support approximately 10,000 users per day.
- Handle concurrent requests without crashing.
- Maintain consistent response times.
- Support stateless backend deployment.
- Support horizontal scaling.

- Operate behind a load balancer.
- Cache static lookup data where appropriate.
- Use optimized SQLite indexes.
- Log server errors without exposing sensitive information.
- Recover gracefully from transient failures.

---

14. Security Requirements

The application shall:

- Validate all user inputs.
- Prevent SQL Injection.
- Prevent Cross-Site Scripting (XSS).
- Prevent Cross-Site Request Forgery (CSRF).
- Sanitize request payloads.
- Use parameterized SQL queries.
- Protect backend endpoints.
- Never expose database credentials.
- Never expose recommendation weights through public APIs.

---

15. Non-Functional Requirements

The application shall:

- Operate without Artificial Intelligence.
- Operate without Machine Learning.
- Operate without OpenAI.
- Operate without Claude.
- Operate without Gemini.
- Operate without Anthropic APIs.
- Operate without external recommendation engines.
- Store all recommendation data locally.
- Produce deterministic recommendations.
- Maintain high availability.
- Support responsive layouts.
- Remain maintainable and extensible.

---

16. Error Handling

The application shall provide meaningful error handling.

Examples include:

- Invalid questionnaire submission.
- Missing recommendation criteria.
- Database connection failure.
- Invalid card comparison request.
- Search returning no results.

Error messages shall be user-friendly and shall not expose internal implementation details.

---

17. Logging & Monitoring

The backend shall log:

- Application startup.
- API requests.
- API errors.
- Database errors.
- Unexpected exceptions.

Sensitive information shall never be written to log files.

---

18. Scalability & Deployment

The application shall support:

- Docker deployment.
- Docker Compose.
- Stateless backend architecture.
- Reverse proxy deployment.
- Horizontal scaling.
- Future migration from SQLite to enterprise databases such as PostgreSQL or MySQL with minimal application changes.

---

19. Future Enhancements

Future releases may include:

- ICICI Bank
- Kotak Mahindra Bank
- HSBC
- IndusInd Bank
- Yes Bank
- IDFC FIRST Bank

Additional future features may include:

- Saved recommendation history.
- User accounts.
- Reward calculators.
- Spending simulations.
- Card update notifications.
- Administrative dashboard.
- Database import/export tools.

---

20. Risks & Assumptions

Assumptions

- Official bank websites provide accurate and current credit card information.
- Credit card information is manually maintained.
- SQLite is sufficient for Version 1 deployment.
- Users provide accurate preference selections.

Risks

- Banks may change benefits without notice.
- Annual fees and reward structures may change.
- Promotional offers may become outdated.
- Manual database updates may be required periodically.

---

21. Success Criteria

The project shall be considered successful if:

- Users can complete the questionnaire without errors.
- The recommendation engine returns deterministic results.
- Every supported credit card is stored in SQLite.
- Recommendation explanations are transparent and accurate.
- Users can compare any two supported cards.
- Search and filtering operate correctly.
- The application performs reliably under the expected workload.
- Docker deployment is successful.
- The architecture supports future expansion without changes to the recommendation engine.
- No Artificial Intelligence, Machine Learning, Large Language Models, or external recommendation services are used anywhere within the application.

Appendix

Supported Technologies

Frontend

- React
- TypeScript
- Tailwind CSS

Backend

- FastAPI **or**
- Node.js Express

Database

- SQLite

ORM

- SQLAlchemy **or**
- Prisma

Deployment

- Docker
- Docker Compose

Version Control

- Git

Prohibited Technologies

- OpenAI
- Claude
- Gemini
- Anthropic APIs
- TensorFlow
- PyTorch
- LangChain
- LlamaIndex
- Pinecone
- ChromaDB
- FAISS
- External recommendation APIs
- AI-powered scoring systems

This concludes the complete PRD. Together with Part 1, it forms a substantially more detailed specification that aligns closely with the comprehensive `instruction.md` and provides clear guidance for implementation.